Q1. Given N tasks, K workers, arr[i] → represents time taken to complete i^{th} task. Find the minimum time to complete all tasks.

* 1 task can be done by 1 worker
* A worker can only do continuous tasks
* workers work parallely

ex ⇒

$w = 3$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 44$ $w_2 \to 12$ $w_3 \to 15$

total → 44.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 24$ $w_2 \to 25$ $w_3 \to 22$

total → 25

∴ Minimise the max^m time taken by any worker

target → $\underbrace{\text{time}}$ minim time so that all tasks are complete

Search space → time

minim ——————————————— maxm

maxm element of arr — sum of all in arr

0

minm ———————————— mid ———————————— maxm

maxm element of arr

sum of all arr

mid

able to complete all tasks with w workers in [mid] time

ans = mid

r = mid-1

not able to complete all tasks with w workers in [mid] time

l = mid+1

1 2 3 -- . 17 18 19 | 20 21 22 23 ——

minm

w=4

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

l	r	m
10	71	40

ans = 40

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 34$ $\qquad$ $w_2 \to 37$

l	r	m
10	39	24

ans = 24

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 24$ $w_2 \to 21$ $w_3 \to 20$ $w_4 \to 6$

$l = 10$ $r = 23$ $m = 16$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 16$ $w_2 \to 15$ $w_3 \to 14$ 16 w_4 X

$l = 17$ $r = 23$ $m = 20$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$w_1 \to 16$ $w_2 \to 18$ $w_3 \to 15$ $w_4 \to 16$ X

$l = 21$ $r = 23$ $m = 22$ ans = 22

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

$l = 21$ $r = 21$ $m = 21$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
3	5	1	7	8	2	5	3	10	1	4	7	5	4	6

X

$l = 22$ $r = 21$

X

ans = 22

```
boolean check( arr, mid, w){        no. of workers given
    worker = 1
    current_work = 0

  for( i=0; i<N; i++){
    if( current_work + arr[i] <= mid){
           current_work += arr[i]
    }else {
           worker++
           curr.work = arr[i]
    }
  }
    if( worker >W)
           return false
    }
    return true
}
```

$$\boxed{\begin{aligned} TC &\Rightarrow O(N\log(\text{sum}[arr])) \\ SC &\Rightarrow O(1) \end{aligned}}$$

H.W   check → Aggressive Cows] → Binary Search
       check → Worker

6 Ques → BS on array → search
floor
freq
localminima
twice | unique
rotated

5 Ques → BS on search space
↓
* sqrt
* subarray size k
* Ath magical no.
* Aggressive cows
* worker

Q2. Find a no. k in a sorted matrix

- sorted row wise
- last element of each row is smaller than 1st ele of next row

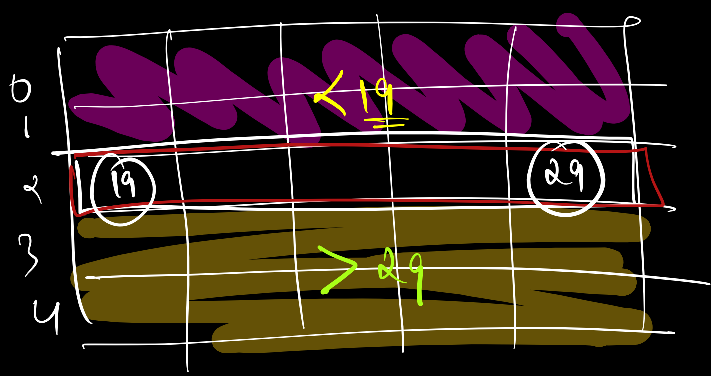
4	9	11
15	17	19
24	26	29

k = 25

Brute force → TC → O(N×M)
↓
O(N log M)

1) find row and then BS.

TC => O(N + log M)



l = 0 r = N-1 = 4

mid = 2

```
if( arr[m][0] <= k
        && arr[m][n-1] >= k) )
        row = mid
        break;
}
else if ( arr[m][0] > k) {
        r = mid -1
else {
        l = mid + 1
}.
```

$$\boxed{\begin{array}{ll} TC \Rightarrow & O(\log N + \log M) \\ SC \Rightarrow & O(1) \end{array}}$$

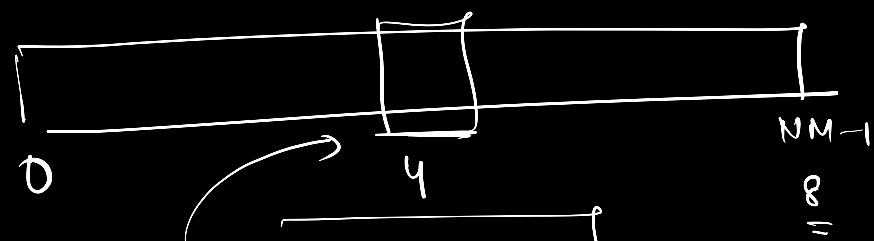
$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 4 & 9 & 11 \\ 1 & 15 & 17 & 19 \\ 2 & 24 & 26 & 29 \end{array} \Rightarrow \boxed{4 \mid 9 \mid 11 \mid 15 \mid 17 \mid 19 \mid 24 \mid 26 \mid 29}$$

	4	9	11	15	17	19	24	26	29
idx →	0	1	2	3	4	5	6	7	8
col →	0	1	2	0	1	2	0	1	2
row →	0	0	0	1	1	1	2	2	2

idx % M ⇒ $\boxed{\text{idx \% 3} \Rightarrow \text{columns}}$

idx / N ⇒ $\boxed{\text{idx / 3} \Rightarrow \text{rows}}$

$$\boxed{\begin{array}{ll} \text{rows} \rightarrow & \text{idx}/N \\ \text{column} \rightarrow & \text{idx} \% M \end{array}}$$



row → 4/3

col → 4%3

l = 0

r = 8

m = 4

if(arr[mid / N] [mid % col])

== k

return [mid / N, mid % col]

else if (> k)

r = m-1

else

l = m+1

TC → $O(\log(NM))$

↓

$O(\log N + \log M)$

SC → $O(1)$

$\log(a \times b)$

$= \log(a) + \log(b)$

HW

Q Given an arr[N], and k.

find kth element in sorted form of arr[N]
(0 based idx)

* modification of i/p arr not allow

* no extra space

k = 4

ex ⇒ 12, 3, 2, 9, 7, 4, 13, 1

sort ⇒ 1 2 3 4 7 9 12 13
0 1 2 3 4 5 6 7

O/p = 7

→ no duplicates ⇒ ans is element having
'k' elements lesser than it.

→ duplicates →

arr[] ⇒ 2 3 6 1 5 6 5 5 5 4

k = 5

sort ⇒ 1 2 3 4 5 $\boxed{5}$ 5 5 6 6
0 1 2 3 4 $\boxed{5}$ 6 7 8 9

$\left[\text{no. of elements} + \text{no. of duplicates} \right]$

→ BS